

U.T. 8: REDES DE ORDENADORES

Objetivos:

- Conocer lo que es una red informática.
- Distinguir los elementos de una red informática.
- Conocer la arquitectura de red TCP/IP

8.1.– SISTEMAS DE COMUNICACIÓN

Un sistema de comunicación es un conjunto de elementos que intervienen en el proceso de intercambio de información y que permiten que las personas puedan comunicarse entre sí.

Por ejemplo, uno de los sistemas de comunicación más antiguo es el correo tradicional y toda su estructura de oficina de correos, carteros, cartas, etc.

Cualquier sistema de comunicación tiene los siguientes componentes:

- **Mensaje:** Contiene la información que se quiere transmitir.
- **Emisor:** Elemento que genera el mensaje.
- **Receptor:** Elemento que recibe el mensaje.
- **Medio o canal:** Es el medio físico que se usa para transmitir la información. Por ejemplo, las señales de humo son un medio o canal.
- **Protocolo:** Es el conjunto de reglas que se siguen para llevar a cabo la transmisión del mensaje.
- **Transductor:** Es un dispositivo encargado de transformar la naturaleza de la señal



8.2.– ¿QUÉ ES UNA RED DE ORDENADORES?

Una red de ordenadores es un sistema de interconexión entre equipos que permiten compartir recursos e información entre ellos.

Se pueden clasificar las redes en distintos tipos en función de su extensión o ubicación:

- **LAN (Local Area Network):** Se llaman así las redes que conectan equipos dentro de un mismo edificio o recinto limitado relativamente pequeño.
- **CAN (Campus Area Network):** Se conocen así a las redes localizadas geográficamente dentro del ámbito de un conjunto de edificios (universitario, oficinas de gobierno, fábricas o industrias) pertenecientes a una misma entidad en un área delimitada en kilómetros.
- **MAN:(Metropolitan Area Network):** Se conocen así a las redes localizadas en el ámbito de una ciudad.

- **WAN:(Wide Area Network):** Se conocen así a las redes localizadas geográficamente en distintas ciudades o incluso países, a cientos de kilómetros una de otra.
- **GAN:(Global Area Networks):** Se conocen así a las redes de alcance global, que están localizadas en todo el planeta.

Atendiendo a la forma en la que están conectados los ordenadores también podemos establecer distintas categorías de redes:

- **Redes sin tarjetas:** Usan enlaces a través de otros puertos (serie, paralelo, USB) para transferir archivos o compartir periféricos.
- **Redes punto a punto:** Comunican dos equipos determinados de forma permanente.
- **Redes entre iguales:** Los ordenadores conectados pueden compartir información con los demás sin que existan roles de cliente o servidor entre los equipos de la red.
- **Redes basadas en servidores:** Una arquitectura cliente/servidor, en la cual el servidor ofrece una serie de servicios a los clientes.

8.2.1.- VENTAJAS DE LAS REDES.

Hoy en día es fundamental casi para cualquier organización disponer de una red. Entre las ventajas de disponer de redes de ordenadores están:

- Compartir periféricos costosos como pueden ser impresoras, etc. Reduciendo de esta forma el coste económico del sistema.
- Compartir grandes cantidades de información de manera que su uso, actualización y manejo sea más fácil.
- Reducir o incluso eliminar la duplicación de trabajos.
- Permitir la utilización de los programas de comunicaciones como el correo electrónico para enviar o recibir mensajes diferentes de otros usuarios de la misma u otras redes.
- Mejorar la seguridad, control de acceso y uso de la información que usa la organización y que es uno de los elementos más importantes de la misma.

8.3.- COMPONENTES DE UNA RED INFORMÁTICA

Para poder tener una red de ordenadores es necesario contar tanto con sus componentes hardware como software.

Componentes hardware:

- Las **tarjetas de red** que se deben instalar en todo ordenador para soportar la conexión a la red.
- Los **cables de conexión** entre los diferentes equipos.
- Los **dispositivos de interconexión** que conectan entre sí los diferentes equipos de la red.
- Los **servidores:** Equipos que ofrecen servicios a los demás equipos conectados a la red.
- Las **estaciones de trabajo:** Cada uno de los ordenadores conectados a los servidores.
- Los **recursos compartidos hardware:** Periféricos interconectados en red a disposición de los usuarios: impresoras, unidades de almacenamiento, etc.

En cuanto a los **componentes software**:

- Los **programas controladores** de tarjetas de comunicaciones y de los equipos periféricos.
- Los **sistemas operativos de red** que permiten la gestión de forma centralizada.
- Los **recursos compartidos software**: Las aplicaciones y ficheros a disposición de los diferentes usuarios.

8.4.– ARQUITECTURA DE UNA RED.

La arquitectura de una red viene definida por tres características que definen la tecnología que se emplea en la red:

- **Topología.** Es la organización de su cableado. Define la interconexión de las estaciones y el camino de transmisión de datos sobre el medio de comunicación.
- **Método de acceso a la red.** El método de acceso define la forma y protocolo mediante el cual cada elemento de la red accede al medio. Generalmente todos los elementos de la red comparten el medio de transmisión.
- **Protocolo de comunicaciones.** Conjunto de reglas perfectamente organizadas y convenidas de mutuo acuerdo entre los participantes en una comunicación, cuya misión es permitir el intercambio de información entre los dos dispositivos, detectando los posibles errores que se produzcan. Estas reglas tienen en cuenta el método para corregir errores, establecer la comunicación, etc.

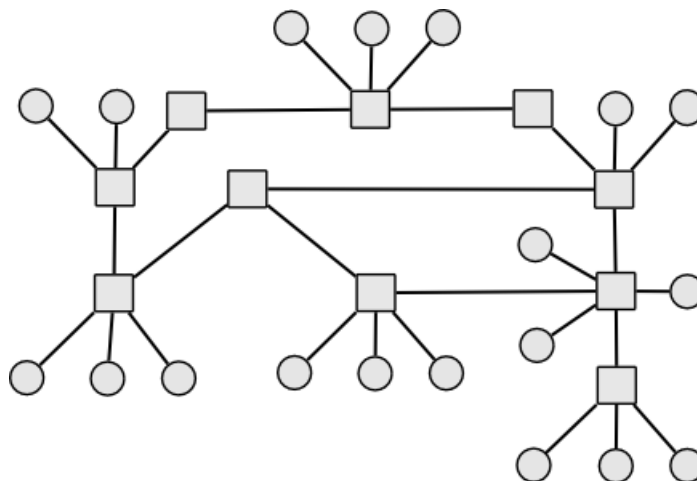
Los problemas más importantes en el diseño de una red de comunicaciones son los siguientes:

- **Encaminamiento.** Es necesario elegir el camino de un mensaje para llegar a destino. usualmente el camino más corto o el que tenga menor tráfico.
- **Direccionamiento.** Se debe identificar el ordenador al que hay que enviar un mensaje.
- **Acceso al medio.** Al compartir el medio de transmisión entre los elementos de una red, Hay que regular el acceso al medio para evitar colisiones.
- **Saturación del receptor.** Un emisor rápido y un receptor lento saturarían al receptor generando pérdida de datos.
- **Mantenimiento del orden.** Hay que mantener el orden de los fragmentos de un mensaje sino resultaría imposible su interpretación.
- **Control de errores.** Se debe revisar si se ha recibido correctamente la información enviada.
- **Multiplexación.** En algunas situaciones se comparte un medio entre comunicaciones que no tienen que ver entre sí. Hay que asegurar la integridad de los mensajes.

8.5.– FUNCIONAMIENTO DE LAS REDES

Una red de computadoras es un conjunto de equipos nodos y software conectados entre sí por medio de dispositivos físicos o inalámbricos que envían y reciben impulsos eléctricos, ondas electromagnéticas o cualquier otro medio para el transporte de datos, con la finalidad de compartir información.

Las redes entre computadores se han estructurado desde su inicio intentado maximizar el número de enlaces entre ellas. Hemos de imaginar una red de computadores como un **grafo** matemático.



Los vértices del grafo son los computadores de la red mientras que las aristas serían los enlaces físicos que los conectan. En un grafo, un camino entre un nodo y otro representa la secuencia de nodos que hemos de cruzar para llegar del nodo origen al destino. En las redes de comunicaciones se intenta maximizar el número de caminos posibles entre nodos para evitar que, aun fallando un número de enlaces, la comunicación entre todos los nodos se vea afectada.

En teoría de redes se suele usar, refiriéndonos a los nodos o computadores los términos de nodo final o nodo enlace. Podemos ver ambos tipos de nodos en la figura, en donde se representan los nodos enlace como cuadrados y los finales como círculos.

- **Nodo final:** Aquel nodo o computador que no posee nada más que un enlace que lo una al resto.
- **Nodo enlace:** Aquel nodo que posee más de un enlace y por el que suele transitar tráfico entre otros nodos.

A medida que las redes de computadores se fueron (durante los años ochenta, sobre todo) uniendo unas a otras se fue creando una estructura más amplia formada por estas redes. De ahí que cuando Internet empieza a tomar forma como red completa se la llamó la **red de redes**.

Para que dos computadores puedan enviarse datos a través de un enlace físico estos deben realizar antes un procesamiento concreto. Dicho de otra manera, ambos deben procesar los datos de una forma concreta o lo que es lo mismo: decidir como codificaran los datos, en que orden los van a enviar, quien enviará primero, etc.

Todos estos acuerdos se toman en un código que ambos computadores ejecutarán a la vez. A este código se le llama **protocolo**. Un protocolo no es más que un programa o librería donde se agrupa el procesamiento que un computador debe realizar para enviar o recibir datos a otro computador que ejecuta el mismo protocolo. Estos protocolos no suelen desarrollarse de forma monolítica. Los diseñadores separan los retos u objetivos que debe enfrentar el protocolo (codificación, confidencialidad de los datos, arbitraje del medio de transmisión, etc.) y una vez decididos estos los separan en capas. A esta forma de trabajar o de diseñar se le llama **Arquitectura por capas**. Para ilustrar esta arquitectura por capas lo ideal es pensar en que cada capa es una clase de Java. Cada capa está programada de forma aislada pensando en solucionar un

problema concreto, pero a la vez, usa a otra capa (la adyacente) para ayudarse a solucionar su problema.

En torno a 1975 en la universidad de Berkley se comenzaron a poner los primeros bloques de lo que hoy conocemos como arquitectura TCP/IP y que es la arquitectura estándar en Internet hoy en día. Esta arquitectura actualmente se compone de cinco capas principales:

- **Nivel Físico:** Es el encargado del transporte propiamente dicho de los datos.
- **Nivel de Enlace:** Da acceso al medio físico a todos los nodos que lo comparten. Es el encargado de arbitrar el uso coordinado del medio de transmisión entre los nodos conectados a él.
- **Nivel de Red:** Calcula el camino más corto entre dos nodos que no comparten medio físico.
- **Nivel de Transporte:** Comparte el acceso a los niveles inferiores a para cada proceso de esa máquina. Pensad que en cada máquina se ejecutan múltiples procesos. Son los procesos los que, en última instancia envían y reciben datos, no las máquinas.
- **Nivel de Aplicación:** No tiene un objetivo particular. En él se agrupan todos los protocolos de carácter específico que usan al nivel de transporte para conseguir enviar datos a procesos de otras máquinas. Hoy en día la mayor parte del tráfico de Internet esta monopolizada por un solo protocolo de nivel de aplicación, el HTTP.

8.5.1.- NIVEL FÍSICO O MEDIO DE TRANSMISIÓN

El medio físico más usado hoy en día en nuestra principal red de comunicaciones, Internet es el medio de cable y el medio aéreo. Para el medio de cable disponemos de varias tecnologías como la fibra óptica, el cable de cobre de par trenzado o el cable coaxial, antiguo y casi desaparecido. El **cable de par trenzado** es con el que estamos más familiarizados en nuestras casas o instalaciones más cercanas. Está formado por un cable en cuyo interior se entrelazan varios pares de cables de cobre. El número de pares de hilos de cobre que forman un segmento de cable de par trenzado depende de la categoría de este.

Actualmente la mayoría de los cables que usamos en nuestras instalaciones son de Categoría 6 o UTP-6. Este tipo de cable es muy barato y permite realizar montajes e instalaciones rápidas y flexibles. Sus conectores (de forma cuadrada y parecidos a los de teléfono) se unen al propio cable sin necesidad de caras herramientas ni soldaduras de ningún tipo.

Para conectar varios equipos usando este cable es necesario crear un segmento de cable por cada equipo (con dos conectores en cada extremo de cada segmento) y unir estos segmentos tanto a cada equipo como a un elemento central llamado concentrador o *hub* o también conmutador o *switch*.

Aunque físicamente parezcan lo mismo el funcionamiento interno de estos aparatos es muy diferente:

Mientras que un *hub* se limita realizar una conexión de los equipos de forma puramente eléctrica, como si todos los equipos estuvieran en el mismo cable, un *switch* es consciente de que equipo está en cada una de sus bocas.

Para distinguirlos se usa un identificador que veremos más adelante llamado dirección MAC. De esta forma cuando un *hub* recibe una trama de un equipo, la replica en todas las bocas mientras que el *switch* solo la replica por la boca donde está la máquina destino de esa trama. Este comportamiento evita las que se conocen como colisión (situación que se produce cuando una máquina quiere enviar algo y hay otra usando su segmento de cable). En caso de usar un *switch* nuestro segmento de cable hasta dicho *switch* solo estará siendo usado si yo envío algo o si me están enviando algo a mí.

El cable de **fibra óptica** es muy usado en la actualidad, pero a un nivel diferente. Suele ser el cableado con el que las compañías proveedoras de Internet nos suministran los datos hasta nuestros domicilios u oficinas y con el que se cablean largas distancias como ciudades, provincias, etc. Permite enviar o recibir a distancias realmente grandes y es el usado incluso para cablear los enlaces que atraviesan los océanos.

Casi la totalidad de enlaces que se usan en Internet hoy en día para enlazar grandes distancias son a través de cables. Para comunicar dos continentes todavía sigue siendo todavía más rentable usar un cable submarino de fibra de cientos de kilómetros que usar un enlace a través de satélites, por ejemplo.

También podemos usar mecanismos **inalámbricos** para realizar enlaces de comunicaciones. Existen multitud de tecnologías para este efecto que varían en la pila de protocolos usadas y no todas son compatibles con TCP/IP (bluetooth, GPRS, Infrarrojos, etc.).

8.5.2.- NIVEL DE ENLACE

El principal objetivo de esta capa es dar acceso a los nodos, al medio de transmisión de forma organizada y coordinada. El medio, debe ser compartido entre varios nodos (porque en caso contrario no habría comunicación de ningún tipo) y su acceso debe estar organizado, arbitrado y secuenciado.

¿Quién envía antes? ¿Qué ocurre si varios nodos quieren usarlo de forma simultánea? Estas cuestiones podrían haberse solucionado usando una entidad aparte, una especie de nodo árbitro o controlador del medio. Pero esta solución habría ocasionado otros problemas ¿Qué pasa si este nodo central falla? Se decidió un mecanismo descentralizado e incorporado a todos los nodos en una capa adyacente a la encargada de enviar los datos o nivel físico.

Este mecanismo se agregó a una capa llamada «Nivel de enlace». La IEEE (Organización de Estandarización Industrial en la Industria Electrónica) agrupó todos estos protocolos o diferentes implementaciones de esta capa bajo la nomenclatura IEEE 802. Así por ejemplo os sonará el protocolo 802.11, el usado en redes inalámbricas de tipo WiFi.

Uno de los más usados es el nivel de enlace utilizado sobre redes de cable de par trenzado. Este protocolo es el denominado **IEE 802.3** y es una estandarización del tradicional protocolo «Ethernet».

El principal objetivo de este protocolo era secuenciar el acceso al medio y evitar solapamientos, esto es, que dos máquinas inyecten datos en el medio de transmisión a la vez. Un envío de un dato no deja de ser una señal eléctrica sobre un cable de cobre. Que dos nodos hagan esta operación produciría una interferencia en la señal y el envío de ambos datos quedaría corrompido.

Para evitar esta situación todos los nodos implementan **el mecanismo CSMA/CD**. Este mecanismo es muy sencillo y efectivo a la vez. Cada nodo está permanentemente «escuchando» el cable de transmisión. Si se detecta una señal eléctrica quiere decir que alguien está enviando y entonces ese nodo no envía nada. Si no detecta nada entonces se realiza el envío porque se entiende que el medio no está siendo usado.

El modelo de capas de una red TCP/IP podemos imaginarlo como un edificio. Así pues, una red de nodos es como un conjunto de edificios, una ciudad, donde tenemos calles que conectan varios edificios.

Cada calle en este ejemplo sería lo que hemos llamado un medio de transmisión (por simplificar, un segmento de cable).

Por las calles solo pueden circular mensajeros que llevan mensajes entre edificios, pero solo un mensajero en cada momento. En ese caso el nivel de enlace es algo así como el conserje que hay en cada planta baja de cada edificio. Él se encarga de mirar cuando la calle está vacía y señalarle al mensajero a que edificio de la calle debe ir para llevar el mensaje.

¿Qué ocurre si el mensaje debe ir al otro lado de la ciudad y no solo a otro edificio de la misma calle? Esto lo solucionaremos en el nivel de red.

El envío de los datos a este nivel se realiza encerrando o encapsulando los datos en tramas o *frames*. La idea es no enviar cada bit de forma aislada por el cable sino agruparlos en unidades o paquetes con una estructura determinada. La información que el nivel de enlace recibe de su nivel superior, el de red, es dividida en estas tramas de tamaño fijo y cada una de ellas se envía al nodo correspondiente.

Cada trama tiene una estructura concreta comenzando por un patrón reconocible para todos los nodos llamado SOF (Start of Frame). De esta forma cuando cada nivel de enlace de cada computador conectado al medio de transmisión escucha o detecta este patrón de bits en el cable sabe que lo que vendrá después es una trama.

A un mismo medio de transmisión podemos tener conectados varios nodos ¿cómo sabe cada nodo si la trama que viaja en ese momento por el medio es para él? Cada trama tiene definido tanto el nodo origen de esta como el nodo destinatario. Esto se hace **identificando a cada nodo que comparte un mismo medio de transmisión** a través de un identificador único llamado dirección MAC.

El identificador o **dirección MAC** es un número único de 48 bits que debe ser diferente para cada nodo que esté conectado a un mismo medio de transmisión.

No debemos confundir este identificador con otro muy conocido en el ámbito de las redes: la dirección IP. Son diferentes y este segundo lo estudiaremos más adelante.

La dirección MAC solo tiene validez a nivel de enlace y aunque se intenta que sea única para cada nodo podría repetirse.

¿Ocurriría algo si dos nodos tienen la misma dirección MAC? No, siempre y cuando no compartan medio de transmisión. Por ejemplo, entre un nodo de Fuenlabrada y otro de Moscú no habría conflicto a este nivel.

Volviendo a la trama, tras el SOF, y las direcciones MAC de origen y destino se ubica la información propiamente dicha del nivel superior (el de red).

Para terminar con el nivel de enlace hablaremos del **protocolo ARP**. Cuando el nivel de red (el nivel conceptualmente superior al de enlace) le pasa al de enlace el paquete que quiere que este envíe a través del medio, lo hará identificando al destinatario usando otra numeración propia del nivel de red: su dirección IP. Como se construye y se gestiona este identificador por ahora no nos preocupa, pero si el nivel de red le dice al de enlace: *Quiero que mandes esto a este nodo* ¿Como hace el nivel de enlace para encontrar la dirección MAC del nodo en cuestión? Esta es la función del protocolo ARP. Sin meternos en muchos detalles, a través de este protocolo un nodo puede preguntar a todos los que comparten su mismo medio de transmisión algo así como *¿Quién tiene la dirección IP xxxxx?*. En ese caso, solo contestará el aludido y al hacerlo usando una trama del nivel enlace ya sabremos la dirección MAC a la que tenemos que enviar la trama que nos pasó el nivel de red.

8.5.3.- NIVEL DE RED

Una vez hemos solucionado el envío y la recepción de datos a través de un mismo medio de transmisión a través del nivel de enlace surge otro problema. ¿Cómo podemos enviar y recibir datos a un nodo que no comparte el mismo medio con el nuestro? Recordad la figura que mostraba una red en forma de grafo. Tomad dos nodos, uno a cada extremo la imagen y comprobad como no comparten un mismo medio o cable. Si queremos enviar un dato entre ambos hemos de enviárselo antes a otros nodos intermedios que harán de intermediario. Algo así como ir pasando el testigo entre varios corredores hasta llegar al nodo final.

Para solucionar este problema del envío de datos entre diferentes medios de transmisión, en lugar de seguir complicando el protocolo de nivel de enlace se decidió crear una capa aparte con sus propios protocolos: El nivel de red. El objetivo del nivel de red es el de encontrar un camino en el grafo o red global entre un nodo origen y un nodo destino que no tienen por qué compartir medio de transmisión.

Anteriormente hacíamos referencia al ejemplo de la ciudad y los edificios. Ahora tenemos en cada edificio una planta encima de la conserjería. En ella se preparan las valijas para ser llevadas a cualquier edificio de la ciudad por lejos que esté. Pero nuestros mensajeros no saben ir más allá de nuestra propia calle ¿Cómo lo haremos? Existen edificios que tienen conexión con varias calles, algo así como puertas a una calle y a otra. Son una minoría, pero en todas las calles existe al menos un edificio así. Si nuestro paquete en realidad debe viajar al otro punto de la ciudad nuestro único objetivo será sacarlo de nuestra calle. Así que al nivel de enlace que está justo por debajo nuestro (en la planta baja) le diremos que mande el paquete a este vecino nuestro con dos puertas.

En el ejemplo anterior, estos nodos o edificios con puertas a dos calles (o redes) son los conocidos como encaminadores o *routers*. Pueden tener puertas o conexiones a más de dos redes incluso. Cuando queremos enviar algo fuera de nuestra red o calle debemos enviárselo a ellos realmente. Una vez que ellos lo reciban (a través de su nivel de enlace) y lo suban a la planta de arriba (a su nivel de red) verán que el paquete no es para ellos realmente, sino que debe ser encaminado o reenviado a través de una de las redes a las que ellos tienen conexión para que llegue, bien a su destino final o bien a otro nodo encaminador similar a ellos, pero con conexión a otras redes más lejanas. Así el paquete va viajando de encaminador a encaminador y cada uno de ellos va escogiendo el camino más apropiado para que el paquete llegue al destino final.

En encaminamiento no se elige directamente en el origen, como hacemos nosotros cuando viajamos en coche, por ejemplo. En ese caso tenemos un mapa y antes de salir ya sabemos toda la ruta que vamos a trazar a nuestro destino. En nuestro caso de encaminamiento, ningún nodo tiene un mapa global de Internet. Ni siquiera los encaminadores. Ellos, digamos que solo tienen un mapa local de sus proximidades y a través de algoritmos pueden especular con cuál de las rutas que ellos conocen en sus proximidades es la mejor para el paquete que les acaba de llegar.

Imaginad que es verano y os habéis perdido en mitad de la provincia de Teruel. Vosotros habéis salido esta mañana de Barcelona y queréis llegar a La Coruña. No sabéis como habéis llegado por una carretera minúscula a un pueblo enano donde solo hay un señor mayor sentado en la plaza. Bajáis la ventanilla y le preguntáis como llegar a La Coruña. El buen señor no ha salido nunca de su provincia y no tiene ni pajolera idea de cómo llegar tan lejos pero dentro de su conocimiento local (su mapa local) os intentará reenviar a lo que él cree que es mejor. Os indica que deis la vuelta y toméis el segundo desvío a la salida del pueblo para tomar una carretera más ancha a Zaragoza. Y allí seguro os sabrán indicar mejor.

En este ejemplo, el buen señor no es más que un router pequeño que desconoce vuestro destino final pero aun así os sabe encaminar lo mejor que puede usando su conocimiento local. Os manda a Zaragoza porque allí tendréis seguro un buen router que conozca mejores rutas. Igual tampoco os mandan a La Coruña directamente, pero os indicarán autopistas para Burgos o Bilbao y ya estaréis más cerca del destino. Así funciona el encaminamiento en Internet. Cada router recibe paquetes y los va reenviando a otros usando su conocimiento local. El paquete irá acercándose al destino hasta que algún router lo conozca directamente y el paquete llegará entonces al nodo final.

De todo este proceso de encaminamiento se encarga el protocolo IP. Uno de los protocolos más importantes de Internet. Actualmente estamos usando mayoritariamente la versión 4 de este protocolo (IPv4) aunque poco a poco en cada vez más nodos se va integrando también la versión 6 de IP o IPv6.

8.5.3.1 DIRECCIONES IP

Para identificar cada nodo en Internet usaremos un identificador único: **la dirección IP**. La dirección IP debe ser única para cada nodo de Internet y no puede repetirse. Está formada por 32 bits que suelen notarse en cuatro grupos de dígitos decimales separados por puntos. Como cada grupo representa a 4 bytes binarios, cada grupo en decimal tomará un valor entre el 0 y el 255 (o lo que es lo mismo entre el 0000 0000 y el 1111 1111).

Como la dirección IP es un identificador de 32 bits ¿cuántos identificadores diferentes podemos formar? Pues el resultado de 2^{32} , un numero mucho menor de identificadores que nodos hay en Internet ahora mismo. Esto es un problema muy serio que se ha logrado subsanar gracias a mecanismos como NAT del que hablaremos más adelante y que permite que muchos equipos compartan direcciones similares.

De cualquier manera, por ahora vamos a simplificar el modelo y vamos a imaginar que este problema no existe y que, tal y como imaginaron los creadores del protocolo cada dirección IP es única para cada nodo. Única para todo internet.

Como se ha mencionado la dirección IP es un conjunto de 32 bits que informan de dos cosas. Una parte de la dirección informa de cuál es tu subred en Internet (algo así como tu calle o tu barrio si volvemos al ejemplo de la ciudad y los edificios de antes) y una parte para identificar al nodo dentro de esa subred. La parte de la dirección IP que indica una cosa y la otra es variable y depende de otro elemento que siempre debe darse junto con la dirección IP y es la **máscara de subred**.

La máscara es otro conjunto de 32 bits, pero con una forma muy peculiar. Todas las máscaras comienzan con un 1 en su parte izquierda y tienen tantos bits a 1 consecutivos como el tamaño del prefijo que indica la subred en la dirección IP. El resto de la máscara serán ceros, también consecutivos hasta llegar a los 32 dígitos. Veamos un ejemplo. En el recuadro tenemos la dirección IP 192.168.0.1 ¿Qué parte de esta dirección indica la subred en la que está el nodo y que parte identifica el nodo dentro de esa subred? Para calcularlo solo tenemos que ver la máscara que la acompaña. En la máscara vemos que hay 24 bits a uno desde la izquierda. Por lo tanto, los 24 primeros bits de la dirección IP serán los que indiquen la subred en la que está el nodo. En este caso la subred de Internet en la que está el nodo en cuestión es la 192.168.0. Dentro de esa subred habrá que identificar al nodo en cuestión (igual que dentro de una misma calle, cada portal tiene su propio número). Los bits restantes de la dirección, en este caso los 8 últimos serán los que identifiquen al nodo en la subred. En este caso es el nodo 1 dentro de la subred 192.168.0.

192	.	168	.	0	.	1
11000000	.	10101000	.	00000000	.	00000001
255	.	255	.	255	.	0
11111111	.	11111111	.	11111111	.	00000000

Con esto anterior debe entenderse que una sola dirección IP junto con su máscara identifica a un nodo (y a la subred en la que está contenido) en todo Internet. Dicho de otra manera: con la información de dirección y máscara podemos encontrar cualquier nodo en Internet por lejos que esté. Es un mecanismo realmente sencillo y a la vez tremendamente efectivo.

8.5.3.2 RANGOS, SUBREDES Y SUBNETTING

De la dirección del ejemplo anterior podemos extraer más información. Por ejemplo, ¿podríamos saber cuántos vecinos hay en la calle del nodo con la dirección y máscara anterior? Es muy sencillo calcularlo mirando el identificador de nodo. Según la máscara suministrada solo los 8 últimos bits de la derecha identifican al nodo. Cualquier nodo de esa red entonces solo podrá ser identificado con 8 bits.

Así que podemos concluir que en la subred donde está ese nodo puede haber solo 2^8 nodos. De aquí podemos extraer la definición de **rango de direcciones** que no es más que un grupo de direcciones consecutivas identificadas como una única dirección y una máscara que indica su tamaño. Para identificar a un rango usaremos normalmente la primera dirección de este. La última del rango suele denominarse dirección de broadcast o de difusión. Para calcular el tamaño del rango es fundamental mirar la máscara y comprobar cuál es el tamaño de la parte de subred y cuál es el tamaño de la parte que identifica al nodo en cada dirección del rango.

Por ejemplo, si preguntamos por el tamaño del rango 195.155.40.0 poco podemos decir. ¿Qué parte de esa dirección tendrán en común todas las direcciones del rango y que parte identificará a cada nodo? Sin saber la máscara no podemos responder nada. Ahora bien, si decimos que el rango es 195.155.40.0 con máscara 255.255.0.0 ya sabemos que los dos primeros bytes de la dirección (195.155) serán compartidos por todos los miembros del rango y los otros dos servirán para identificar a cada nodo. Así que este rango tiene una capacidad de 2^{16} direcciones o nodos.

Dado entonces una dirección de inicio de rango y su máscara, es fácil de entender cómo podemos variar el tamaño de este modificando su máscara. Añadir bits a 1 a la máscara es como ampliar el *zoom* de una foto, estrechas el campo y salen menos objetos. Quitar bits a 1 a la máscara es similar a todo lo contrario, ampliar el campo para que salgan más cosas. Incluso podríamos llegar a partir un rango inicial creando dos mitades, por ejemplo.

Vamos a imaginar que partimos de un rango inicial como el que tenemos definido en el siguiente recuadro:

Rango inicial:	192	.	168	.	0	.	0
	11000000	.	10101000	.	00000000	.	00000000
Máscara inicial:	255	.	255	.	255	.	0
	11111111	.	11111111	.	11111111	.	00000000

Este rango tiene un tamaño de 2^8 direcciones debido a que solo identifica a cada nodo con el último byte de la dirección (el de más a la derecha). Así por ejemplo las direcciones 192.168.0.22 o 192.168.0.112 serían vecinas y pertenecerían a este rango, pero la dirección 192.168.8.28 no.

Vamos a imaginar ahora que queremos partir el rango en dos mitades. Para ello simplemente hemos de incluir un 1 más a la máscara. Esto como es obvio acortará la parte que identifica a cada nodo y por ello, en cada mitad de nuestro rango entrarán menos nodos.

Nueva máscara:	255	.	255	.	255	.	128
	11111111	.	11111111	.	11111111	.	10000000
Rango 1:	192	.	168	.	0	.	0
	11000000	.	10101000	.	00000000	.	00000000
Rango 2:	192	.	168	.	0	.	128
	11000000	.	10101000	.	00000000	.	10000000

Vemos ahora que la máscara ha quedado definida como 255.255.255.128 debido a que hemos añadido ese uno en el último byte. Ahora entonces los rangos ya no serán de 256 elementos sino de la mitad cada uno de ellos (de 128). Para ver mejor esto voy a detallar las direcciones límite de cada rango antes y después de la partición indicando en binario el último byte para entender mejor el ejemplo:

```

192.168.0.0    (00000000)
192.168.0.1    (00000001)
Rango inicial: ....
192.168.0.254  (11111110)
192.168.0.255  (11111111)

```

```

-----

192.168.0.0    (00000000)
192.168.0.1    (00000001)
Rango final 1: ....
192.168.0.126  (01111110)
192.168.0.127  (01111111)

```

```

192.168.0.128  (10000000)
192.168.0.129  (10000001)
Rango final 2: ....
192.168.0.254  (11111110)
192.168.0.255  (11111111)

```

Fijaros bien en el último byte en binario y observar cómo va evolucionando para entender bien el mecanismo (podéis ayudaros de calculadoras binarias como la de Windows para realizar las conversiones más rápidas). El resultado final ha sido convertir un rango de direcciones en el que nos cabían 256 direcciones (el original) en dos en los que nos caben la mitad, 128 direcciones. Añadiendo únicamente un bit a uno a la máscara en su bit número 25 (empezando a contar desde la izquierda).

Una vez entendido el mecanismo os podéis preguntar por la utilidad de esto. Antiguamente, cuando todavía quedaban direcciones IP libres que eran asignadas por los proveedores de Internet a sus clientes, estas eran comercializadas por rangos de mayor o menor tamaño. Si tu como cliente adquirirías un rango para tu empresa de por ejemplo 256 direcciones (como el del ejemplo) y te interesaba montar dos subredes por motivos de infraestructura (por ejemplo, para separar dos departamentos en tu empresa) este mecanismo te permitía hacer esa maniobra. De no poderse hacer esto hubiera sido necesario que adquirieras dos rangos, pagando bien caro por ellos, por cierto, para cada uno de tus departamentos y desaprovechando así muchas más direcciones.

Además de la forma de notación clásica de las máscaras de subred tenemos otra notación más abreviada que es la llamada **notación CIDR**. Esta notación permite dar la información del nombre de rango y máscara en menos dígitos indicando tras la dirección IP del rango una barra inclinada '/' y el número de bits a 1 que tiene la máscara. A continuación, varios ejemplos usando una notación y su equivalente:

Dir ===	Máscara =====	CIDR =====
192.168.20.0	255.255.255.0	192.168.20.0 / 24
10.10.0.0	255.255.0.0	10.10.0.0 / 16
132.195.14.128	255.255.255.128	132.195.14.128 / 25

Una vez entendido el sistema de direccionamiento que usa el protocolo IP para identificar tanto a grupos de nodos como a nodos individuales dentro de Internet vamos a detallar el mecanismo que se usa para encaminar paquetes de datos desde un origen a un destino.

8.5.3.3 ENCAMINAMIENTO

Cuando un router recibe un paquete de datos y lo sube desde su nivel de enlace a su nivel de red puede darse cuenta de que realmente no era para él. A pesar de que la trama venía con su dirección MAC como dirección destino, lo que venía dentro era un paquete del protocolo IP (estos paquetes se denominan datagramas) y en este paquete venía indicado la dirección IP final hacia la que va la información. En ese momento el router sabe que el paquete debe volver a ser inyectado en la red para que continúe su camino, hacia otro router o quizás a su destino final.

Volvamos al ejemplo de las calles y los edificios. Ahora estamos en un edificio que tiene puertas a tres calles diferentes. Un mensajero ha traído la trama desde una de las calles a nuestro edificio. Lo subimos al piso de arriba y allí se dan cuenta de que el paquete no es para nosotros realmente, así que los operarios de la primera planta en lugar de subirlo hacia el piso superior construyen un nuevo paquete para que este vuelva a ser llevado por otro mensajero. ¿Por cuál de las puertas que nos conectan a las tres calles ha de enviarse el paquete? Seguramente por la misma que ha venido no. Pero aún nos quedan dos. Aquí entra en juego el algoritmo de encaminamiento. Este algoritmo revisa lo que llamábamos al principio del tema «el conocimiento local» del router y decide la puerta/calle por la que sale el paquete, y dentro de esa calle, a quién se lo vamos a enviar.

La decisión de como encaminar un paquete se hace mirando la **tabla de encaminamiento** de nuestro router. Cada router posee una tabla en donde guarda su conocimiento local sobre la zona que tiene cerca. Como esté construida esta tabla depende del sistema operativo de nuestro router. Nosotros vamos a usar una simplificación de tabla de rutas que se parece a las más comunes (Linux, Cisco, etc.) pero como digo no existe en ningún sistema tal y como la vamos a detallar. Esta tabla de rutas va a estar compuesta por cuatro columnas:



- La dirección IP de la **red destino** a la que va el paquete y que identificará a la ruta.
- La **máscara** que acompaña a esa dirección IP
- El **siguiente salto** o *gateway* hacia el que irá el paquete. Será la dirección IP del nodo hacia el que enviaremos el paquete.
- La **interfaz física** por la que saldrá el paquete de nuestro sistema. Los routers tienen varias interfaces físicas (tarjetas de red) que los conectan a las diferentes redes a las que dan soporte (las calles en el ejemplo de los edificios).

En el esquema tenemos dos subredes conectadas por un solo router: La subred de la izquierda con identificador 201.124.10.0 con máscara de 24 bits o lo que es lo mismo 255.255.255.0 y la subred de la derecha con identificador 198.65.1.0 con máscara también de 24 bits. Ahora imaginemos que un equipo de la red de la izquierda quiere enviar un paquete a un nodo de su misma subred ¿Sabe cómo hacerlo? Si. No necesitará al router e inyectará el paquete en su misma subred y su vecino lo detectará, lo recogerá y lo subirá hacia sus niveles superiores. Pero ¿qué pasa si este mismo equipo quiere enviar un paquete a un nodo que no está en su subred? En ese caso deberá mandárselo al router.

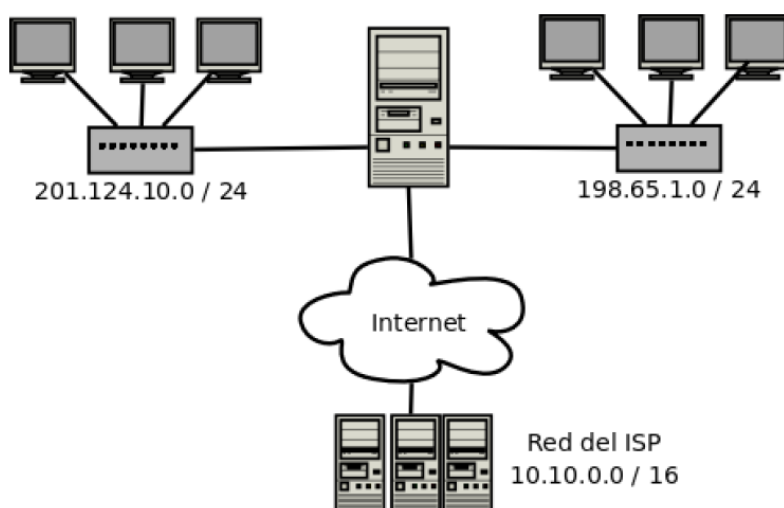
Tabla de encaminamiento del router de la figura 2

Red destino	Máscara	Sig. Salto	Interfaz física
=====	=====	=====	=====
201.124.10.0	24	0.0.0.0	eth0
198.65.1.0	24	0.0.0.0	eth1

Básicamente cada equipo de las dos subredes está configurado de la siguiente manera: si el nodo destino está en su misma red le enviarán el paquete directamente, sin intermediarios. En caso contrario deben enviarle el paquete a su *puerta de enlace*. La puerta de enlace es el equipo que se encarga de encaminarte los datos hacia Internet u otras redes. Básicamente, **si no sabes encaminar un paquete debes dárselo a tu puerta de enlace**. Tenemos entonces a un equipo origen de la red de la izquierda, por ejemplo, el 201.124.10.15 que le quiere enviar un paquete de datos a 198.65.1.22. ¿Está el destino en su red? Claramente no, así que debe enviárselo al router y dejar que él se encargue.

El router entonces recibirá el paquete (con destino 198.65.1.22). Lo abrirá, lo subirá hasta su nivel de red y verá que no es para él y procederá a encaminarlo. ¿Coincide la IP destino con la primera ruta de la tabla? No, porque esa ruta nos lleva a la red de la izquierda, que es de donde viene el paquete así que esa no puede ser la ruta correcta. ¿Coincide la IP destino con la segunda ruta? Vemos que sí. La IP 198.65.1.22/24 pertenece a la red 198.65.1.0/24 así que es la ruta correcta. Entonces sacaremos el paquete por la interfaz de red llamada eth1 y lo enviaremos a la dirección 0.0.0.0. Esta extraña dirección significa que no debemos enviarle el paquete a otro router porque el destino ya se encuentra en esa red. Dicho de otra manera, el paquete no debe de dar más saltos porque ya ha llegado a su destino.

Ejemplo de enrutamiento con Internet



Vamos a incluir ahora una modalidad de encaminamiento algo más completa. En la figura superior tenemos un esquema similar al primero, pero en el que el router tiene una ruta más. Esta ruta será la que conecta a las dos anteriores con Internet. ¿Como se configura esta tercera ruta?

Si nos fijamos en la tabla de rutas anterior vemos que se ha añadido una tercera fila. Esta entrada se usa cuando el router recibe un paquete que no es para él, pero tampoco es para ninguna de las rutas anteriores. Se puede traducir como «para cualquier otra ruta usaremos esta».

Imaginemos ahora que el equipo de la red de la izquierda que usábamos en el ejemplo anterior, el 201.124.10.15 quiere enviar un paquete al nodo con dirección 83.103.10.155. El paquete de datos destino, al no estar en su subred será enviado al router igual que se hacía antes. El router lo procesa y verá que la IP destino no está ni en la red de la primera ruta, ni en la red de la segunda ruta. En ese caso tomará la última ruta que al tener la entrada 0.0.0.0 es la que vale para cualquier caso. De esta forma el paquete irá a la IP 10.10.0.1 a través de su interfaz física eth2.

Tabla de encamamiento del router de la figura 3

Red destino =====	Máscara =====	Sig. Salto =====	Interfaz física =====
201.124.10.0	24	0.0.0.0	eth0
198.65.1.0	24	0.0.0.0	eth1
0.0.0.0	0.0.0.0	10.10.0.1	eth2

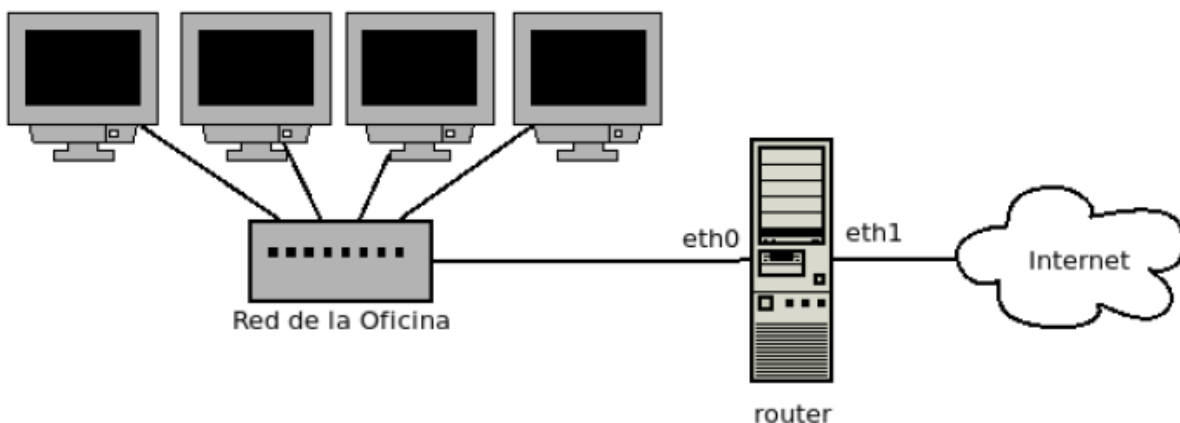
Aquí vemos como un router envía un paquete a otro (su *puerta de enlace*) porque no conocía la dirección destino hacia donde iba el paquete. (Esto es similar a lo que nos hacía el hombre que nos encontrábamos en la plaza de aquel pequeño pueblo al que llegábamos desorientados). Este segundo router seguramente sea uno propiedad del proveedor de internet o de la compañía telefónica que conecta a Internet al usuario del ejemplo. Vamos a solucionar ahora un problema clásico de encaminamiento. El esquema de este ejercicio se parece mucho al que tenemos la mayoría en nuestras casas. Por un lado, a la izquierda la infraestructura de nuestra casa con uno, dos o más ordenadores. En el centro el router de conexión, que es el que nos ofrece el operador ya configurado cuando nos abonamos a su servicio. A la derecha la llamada «red pública» o lo que es lo mismo Internet. Allí encontraremos los routers de nuestro operador y todas las demás máquinas de Internet.

8.5.3.4 EJEMPLO SOLUCIONADO DE UN PROBLEMA DE ENCAMINAMIENTO

Dado un esquema similar al de la figura, donde vemos una serie de equipos conectados (aproximadamente 50 equipos) a un conmutador o *switch* queremos conectarlos entre ellos y a Internet través del router que aparece en el centro de la imagen. Se pide entonces lo siguiente:

1. Configura las direcciones IP y las máscaras de todo el esquema
2. Configura la tabla de encaminamiento del router para ofrecer la conectividad propuesta

Esquema de red de la oficina.



Solución:

Apartado 1)

Primero buscamos un rango de direcciones que nos valga para el escenario propuesto. Cualquier rango de direcciones con máscara de 24 bits (255.255.255.0) nos vale de sobra. Estos rangos ofrecen hasta 256 direcciones por lo tanto nos vale. Así que usaremos, por ejemplo, el rango 192.168.0.0 / 24

Dejamos la primera dirección del rango sin usar para nombrar a la red en la tabla de encaminamiento (la dirección 192.168.0.0). La siguiente dirección, la 192.168.0.1, la asignaremos a la interfaz del router que está conectada hacia la izquierda. Daros cuenta de que el router necesitará dos direcciones IP, una por cada interfaz. La que está hacia la izquierda pertenece a la subred de la izquierda y debe tener una dirección IP válida de esa subred. Así que el router, en su interfaz eth0 tendrá la dirección ip 192.168.0.1.

Las siguientes direcciones se las daremos los equipos. Si, como nos dicen, son 50 equipos usaremos desde la 192.168.0.2 hasta 192.168.0.52.

La otra interfaz del router es la que conecta con la subred que gestiona nuestro proveedor de Internet. Nosotros ahí no podemos tocar nada ni elegir nada. Normalmente será el operador quien nos dirá que rango está usando a ese lado del router.

Apartado 2)

La primera ruta se ha construido siguiendo las directrices explicadas antes. La última ruta, que es la usada por el router si el paquete no va a la anterior, tiene como puerta de enlace la dirección X.X.X.X. Esta es una forma de decir que no nos interesa esa IP. Será una IP que nos comunique nuestro operador y ya está.

192.168.0.0	255.255.255.0	0.0.0.0	eth0
0.0.0.0	0.0.0.0	X.X.X.X	eth1

8.5.4.- IPV6 Y NAT

Anteriormente, cuando hemos hablado de la longitud de las direcciones IP actuales, de 32 bits, no hemos comentado la problemática que se viene arrastrando desde hace casi 15 años. Esta longitud limita el número de direcciones máximas que se pueden formar a 2^{32} direcciones. Este número parecía casi ilimitado cuando se diseñó la versión actual del protocolo IP, en los años 80, pero actualmente nos hemos dado cuenta de que es totalmente insuficiente. Hoy en día, y desde hace casi más de diez años hay muchos más dispositivos conectados a Internet que direcciones IP disponibles. ¿Como es esto posible si, como se ha dicho, cada dispositivo debe tener una dirección única?

El principal mecanismo que se ha usado para «parchear» esta falta de direcciones es un **mecanismo llamado NAT** (*Network Address Translation* en inglés). Este mecanismo, si está activado en un router permitirá que con una sola dirección IP asignada a una de las interfaces del router pueda funcionar toda una red, del tamaño que sea, conectada al otro lado y que pueda estar usando direcciones que no son válidas fuera de esa red.

Imaginad que tenemos una gran red de equipos: por ejemplo, la red del Jovellanos con casi 500 equipos entre todos los edificios. En lugar de tener que usar 500 direcciones únicas en Internet pedimos a nuestro proveedor de Internet solo una dirección IP. Esta se le asigna a la interfaz del router que nos conecta a Internet y es la que se conoce como IP pública. Hacia el otro lado tendremos toda la red del instituto que obviamente funciona con direcciones IP pero son «de mentira». Suelen tener la forma 192.168.xxx.xxx y no son direcciones únicas. Cuando un paquete sale del instituto hacia Internet, el router cambia la dirección origen de este (una de las de mentira) y pone la suya. De esta forma, para el resto de Internet los 500 ordenadores del Jovellanos están funcionando con una sola dirección IP. La pública del router. ¿Qué ocurre si desde fuera queremos enviar un paquete a un equipo del instituto? Habrá que enviarlo a la dirección IP pública del router que es la única que conocemos desde fuera. Pero entonces ¿cómo sabrá el router que el paquete no es para el realmente si la dirección de destino final es la suya? Volveremos a esto en el nivel de transporte cuando hablemos de los *puertos*. Además de NAT desde hace una década más o menos se está implantando poco a poco una **nueva versión del protocolo IP llamada IPv6**. Este protocolo, entre otras mejoras, permite identificadores mucho más largos (de hasta 128 bits).

8.5.5.- NIVEL DE TRANSPORTE

Una vez que hemos explicado y detallado el nivel de red y su principal función, la de encontrar rutas, vamos a desarrollar el siguiente nivel en la pila TCP/IP: el nivel de transporte. Sus principales funciones son la de comunicar procesos (ya no máquinas como el de red sino procesos) y mantener esta comunicación con un flujo óptimo en cada momento y libre de errores.

Hemos de pensar que hasta ahora hemos estado todo el rato hablando de comunicar máquinas. Pero como sabemos ya de temas anteriores, las máquinas no son las que se comunican realmente, son los procesos que ejecutan dentro de ellas. Además, dentro de una máquina normalmente pueden estar siendo ejecutados numerosos procesos que pueden estar queriendo comunicarse con otros tantos de otras máquinas. Así que un primer problema al que nos enfrentamos es el de tener que **compartir el acceso a la red de una sola máquina para todos los procesos** que la quieren usar. Imaginad también que un paquete de datos llega a una máquina. El destinatario real no será la máquina en si sino un proceso que ejecuta dentro de ella. ¿Como sabremos a que proceso va dirigido este paquete? Para etiquetar los paquetes e indicar a que proceso van dirigidos en este nivel de TCP/IP aparece un elemento nuevo, **el puerto**.

Seguramente habréis oído hablar antes de los puertos. Los puertos no son más que etiquetas numéricas que se asocian a cada proceso que quiere usar la red para comunicarse. Si un proceso quiere usar la red se le asigna un número entero, entre el 1 y el 65536 (2^{16}), y ese será su puerto durante toda la sesión de trabajo y hasta que decida dejarlo libre. La idea ahora es que, desde el nivel de transporte de una máquina, cuando queremos enviar un paquete de datos a otro proceso ya no basta con indicar la dirección IP de la máquina destino, además tenemos que indicar a que puerto de esta máquina queremos enviar el paquete. Dicho de otra manera, con un puerto y una dirección IP identificamos a un proceso concreto en toda Internet.

Volviendo al ejemplo de las calles y los edificios de oficina. Imagina que quieres enviar un paquete a otro edificio. No te vale solo con la dirección postal del edificio, además debes incluir el despacho, la extensión o el cargo en el que está el destinatario real o se quedará en la conserjería hasta que alguien pregunte por él. Este dato sería algo así como el puerto en una conexión de red.

En el nivel de transporte aparecen dos roles en una comunicación. Siempre tendremos un proceso que debe actuar como **cliente** y uno que actuará como **servidor**. El cliente es siempre quien inicia la comunicación enviando un primer paquete de datos al proceso servidor que estará siempre a la espera. Cuando el proceso servidor recibe este paquete de datos, lo procesará y contestará al cliente con una respuesta. Este mecanismo de envío-respuesta puede repetirse tantas veces como queramos. Como es el cliente quien debe iniciar siempre la comunicación y es imprescindible que este conozca dos datos fundamentales del servidor: su dirección IP y su puerto. O el cliente conoce estos datos a priori o no podrá iniciar comunicación él. ¿Como se almacenan estos datos en nuestros equipos actuales? ¿Existe algo así como una agenda mundial de direcciones IP y puertos en la que podemos consultar? Resolveremos estas dudas un poco más adelante, en el nivel de aplicación.

Para entender bien el concepto de cliente y servidor podemos imaginarnos un bar con muchos clientes. El camarero se encuentra tras la barra en espera de recibir peticiones de los clientes que pueblan el bar. Son los clientes quienes inician siempre la comunicación con el camarero acercándose a la barra y pidiéndole una consumición. Este es un buen ejemplo para entender la relación entre clientes/servidor porque podemos reflexionar más en profundidad sobre situaciones que se dan en escenarios reales en sistemas con procesos clientes y servidores ¿El camarero debe atender otras peticiones mientras está sirviendo a un cliente concreto o debe ir solo de una en una hasta que no sirva una consumición no escuchará peticiones de otros clientes? ¿Qué ocurre si se multiplica el número de clientes con un solo camarero en la barra? ¿Qué ocurrirá si aumentamos el número de camareros en la barra? ¿Y si este número de camareros es demasiado grande? Estas preguntas en relación con clientes y servidores se escapan del temario del curso, pero a donde quiero llegar es que, para entender la relación entre procesos y servidores, el ejemplo del camarero y los clientes puede ayudaros mucho.

8.5.5.1 PROTOSCOLOS TCP Y UDP

En el nivel de transporte tenemos dos protocolos fundamentales. Estos son TCP y UDP. Estos protocolos son clave a la hora de comunicar procesos de forma independiente entre máquinas y de controlar el flujo de mensajes entre ellas. No vamos a extendernos mucho en ambos pues son tremendamente complejos. Lo más importante que destacaremos de ambos es una diferencia clave: la fiabilidad.

El protocolo TCP es totalmente fiable, pero esto ¿qué quiere decir?. Si le damos un bloque de bytes al nivel de transporte para que lo envíe usando TCP y le suministramos además una dirección IP destino y un puerto, el nivel de transporte troceará el bloque de bytes en varios paquetes TCP y los enviará. Pasado un tiempo sabremos si el bloque completo ha llegado o no. Eso quiere decir que es fiable. TCP nos **asegura si la información ha llegado y además si ha llegado en orden y libre de errores**. En caso contrario nos informa de que ha habido errores. Ser fiable implica que siempre tendremos certeza de cómo ha ido el envío, bien o mal. Nunca tendremos lugar incertidumbres. Para conseguir esta fiabilidad TCP implementa un complejo mecanismo de intercambio paquetes entre origen y destino que provocan que la latencia total del envío se dilate. Enviar con garantías es muy útil pero no es gratis y nos costará más tiempo. TCP tiene dos tipos de paquetes: los paquetes de tipo SYN y los paquetes de tipo ACK. Los primeros son los encargados de transportar datos mientras que los segundos se usan para indicar a la otra parte que hemos recibido parte de esos datos.

Cuando tenemos un escenario en el que queremos priorizar la velocidad de transferencia frente a la fiabilidad podemos usar un protocolo hermano llamado UDP. Este protocolo no nos aporta la fiabilidad del anterior y por lo tanto no podemos asegurar que, si enviamos algo, esto llega al destino total o solo parcialmente. Sin embargo, lo que si aseguramos es que el envío se hará de forma mucho más rápida.

8.5.5.2 PROCESOS SERVIDORES EN REDES CON NAT

Anteriormente hemos hablado del sistema NAT. Este sistema se activa en un router para indicarle que el funcionará como intermediario entre Internet y todos los equipos que tiene tras él. Esto permite que todos esos equipos funcionen con direcciones IP «de mentira». Estas direcciones pueden estar repetidas en otras redes y no se deben adquirir comercialmente a nuestro proveedor. Esto se hace de esta manera porque hoy en día los proveedores no pueden vender ya más direcciones IP válidas pues hace años que se terminaron.

Tenemos entonces redes que pueden ser muy grandes (imaginad la red del instituto o incluso redes mayores) que se ven desde fuera como un solo equipo y una sola dirección IP. Cuando desde esas redes se envía un paquete de datos al exterior, desde fuera el nodo que aparece como nodo origen es el único que tiene una dirección IP válida, esto es el router. Pero ¿qué ocurriría si quisiésemos iniciar una conexión contra un proceso que está en una red NAT? Imaginad que estamos en nuestras casas y queremos conectarnos a un servidor que hemos arrancado en el instituto. Para conectarnos a él, como ya se ha dicho necesitamos tanto la dirección IP como el puerto. Pero la dirección IP del nodo en el que está arrancado el servidor es «de mentira». La única dirección IP de verdad que tiene todo el instituto es la dirección «pública» del router. Si mandamos un paquete a la dirección IP del router y a un puerto concreto ¿Como sabrá el router que ese paquete no es para él, sino que es para un nodo que se encuentra tras él?

Lo que tenemos que hacer en estos casos es configurar el router para indicarle que todo lo que le llegue a su dirección IP y a un puerto en concreto realmente no es para él, sino para un equipo de dentro de la subred a la que él está conectando. Crearemos entonces reglas en una tabla donde marcaremos tantas **redirecciones por puerto** como queramos:

- si te llega algo al puerto 80 => lo reenvías a 192.168.10.10 puerto 80 - si te llega algo al puerto 21 => lo reenvías a 192.168.10.20 puerto 250

A esta práctica se la llama **port forwarding** y la conocemos popularmente como **abrir puertos**. Cuando quieres arrancar un proceso servidor en una red NAT y deseas que gente del exterior se conecte a ese servidor debes abrir el puerto en el router para que las conexiones entrantes le lleguen a ese servidor. Podéis pensar en escenarios en los que vosotros habréis tenido que abrir puertos en vuestros routers (pues las redes de vuestras casas son redes NAT). A continuación, comento varios ejemplos que me vienen a la cabeza:

- Usar programas de intercambio como Emule que requieren tener un servidor funcionando en tu propia máquina.
- Crear partidas de videojuegos multijugador en tu propio equipo y dejar que amigos tuyos se conecten desde sus casas.
- Arrancar un servidor de páginas web en tu propia casa para alojar tu propia página personal.

8.5.6.- NIVEL DE APLICACIÓN

Para terminar de desarrollar la pila de protocolos TCP/IP hemos de concluir explicando la capa de mayor nivel de abstracción: la capa o nivel de aplicación. Esta capa no tiene un objetivo tan concreto como las anteriores.

A pesar de que todos los días usamos varios protocolos de nivel de aplicación en nuestro uso cotidiano de Internet este número ha ido decreciendo debido a que uno de ellos ha ido tomando importancia por encima del resto. Este es **el protocolo HTTP**. Este protocolo pensado en su inicio como un sistema de intercambio de ficheros de marcas HTML es hoy en día el mayoritario y fundamental en el uso diario de Internet. Fijaros si este protocolo es importante para la industria del software hoy en día que todo vuestro perfil laboral como *desarrolladores de aplicaciones web* gira en torno a él. Este protocolo es el hilo de comunicación principal que mantiene todos los elementos de una aplicación web unidos y funcionando. Está formado por mensajes de texto plano que viajan dentro de paquetes TCP. Estos mensajes se llaman peticiones HTTP. El servidor recibirá cada petición y generará una respuesta HTTP para contestar al cliente. Para crear una petición HTTP debemos conocer de antemano la dirección IP del servidor al que queremos lanzar esta petición. Normalmente a los humanos nos cuesta recordar series de números, pero sin embargo no cuesta menos memorizar nombres. Es por esa razón que desde un principio se pensó que se iba a necesitar un mecanismo o agenda global donde en un lado tuviéramos nombres fácilmente recordables y en otras direcciones IP asociadas a estos nombres.

Pensar en una base de datos general y compartida por todos para todas las direcciones IP de Internet es algo complicado. ¿Como podemos mantener esa base de datos actualizada? ¿Como la sincronizamos entre todos cada vez que haya un cambio? Para eso se construyó otro protocolo de nivel de aplicación crucial en Internet hoy en día: el **protocolo DNS**. Este protocolo, simplificando mucho su funcionamiento, divide los nombres en zonas (.com, .es, .org, etc.) y cada zona se le asigna a un servidor o máquina. Esta zona solo debe responsabilizarse de un número concreto de direcciones y mantenerlas actualizadas. Así se consigue un sistema escalable a toda Internet.